DEEP LEARNING LAB • CODING ASSIGNMENT

The Storyteller's Apprentice

Text Generation with Transformers

50 marks (+4 bonus) • 3 hours • One notebook, six tasks

> *The lamp in the Wordhaven library has gone dark; the old storyteller has retired. You will build an apprentice: a small transformer that reads thousands of bedtime stories and learns to tell new ones. Then you play detective and find out what it remembers, which of its heads does the remembering, and what happens when you take away its sense of order.*

What You Are Given

- The lab folder contains `data/TinyStories-valid.txt`, a plain-text file of short children's stories. It is the only file you need. Do not modify it.
- No internet access is needed and no pretrained models or checkpoints are used. You build every component yourself, including the tokenizer and vocabulary, so everything your model depends on is created in your own notebook.
- Use a GPU if your machine has one (`"cuda" if torch.cuda.is_available() else "cpu"`). Everything is sized to also run on a CPU.

Fixed Settings

Every setting below is fixed. Do not change any of them unless a task explicitly tells you to.

1. Reproducibility

At the very top of the notebook, before any other code, set `random.seed(42)`, `numpy.random.seed(42)` and `torch.manual_seed(42)`. In addition, call `torch.manual_seed(42)` immediately before you construct each model, and give every training run a fresh training DataLoader whose shuffling uses `torch.Generator().manual_seed(42)`.

2. Loading and splitting the stories

Use exactly this code:

```
with open("data/TinyStories-valid.txt", encoding="utf-8") as f:
    text = f.read()
text = text.replace("\u2019", "'").replace("\u2018", "'")
stories = [s.strip() for s in text.split("<|endoftext|>")]
stories = [s for s in stories if s]        # drop empty pieces
stories = [s for s in stories if len(tokenize(s)) <= 190]   # whole stories only
train_stories = stories[0:8000]
val_stories   = stories[8000:8500]
test_stories  = stories[8500:9000]
```

The second filter keeps only stories that fit completely into the model (at most 190 tokens), so no story is ever cut off and every story keeps its real ending. It uses `tokenize` from setting 3, so define that function before you run this code.

3. Tokenizer

```
def tokenize(s):
    return re.findall(r"[a-z']+|[0-9]+|[^\sa-z0-9]", s.lower())
```

4. Vocabulary

- Count every token of every training story (each story is complete; nothing is truncated). Use the training split only.
- Keep tokens that occur at least 5 times.
- IDs 0 to 3 are reserved: `<pad>` $= 0$, `<bos>` $= 1$, `<eos>` $= 2$, `<unk>` $= 3$.
- The kept tokens get IDs 4, 5, 6, ... in order of count, highest first, with ties broken alphabetically: `sorted(kept, key=lambda t: (-count[t], t))`.
- Build `stoi` (token to ID) and `itos` (ID to token). The vocabulary size V includes the 4 special tokens.

5. Encoding

For each story, map all of its tokens to IDs (unknown tokens become 3). After the filter in setting 2 a story has at most 190 IDs, so nothing is cut. Then build

$$x = [1] + \text{ids} + [2], \qquad \text{right-padded with 0 to length 192.}$$

The model input is `inp = x[:-1]` and the target is `tgt = x[1:]`, both of length 191. The padding mask is `inp != 0` (`True` = real token).

6. DataLoaders

Wrap the `(inp, tgt)` tensors in a `TensorDataset`. Batch size 64. The training loader uses `shuffle=True` with the seeded generator from setting 1. Validation and test loaders use `shuffle=False`. Keep the last smaller batch (`drop_last=False`).

7. Model architecture

Part	Exact specification
Token embedding	`nn.Embedding(V, 128)`
Positional embedding	`nn.Embedding(191, 128)`, looked up with positions $0, 1, \ldots, T-1$
Input to the blocks	$h = \text{Dropout}(0.1)\,(\text{token emb} + \text{positional emb})$
Number of blocks	2, applied one after the other
One block	$h \leftarrow h + \text{Attn}(\text{LN}_1(h))$, then $h \leftarrow h + \text{FFN}(\text{LN}_2(h))$; each LN is `nn.LayerNorm(128)`
Attention	`CausalSelfAttention` from Task 2 (4 heads of 32 features), followed by `Dropout(0.1)` on its output
Feed-forward (FFN)	`Linear(128, 256)` $\rightarrow$ GELU $\rightarrow$ `Linear(256, 128)` $\rightarrow$ `Dropout(0.1)`
Output	final `nn.LayerNorm(128)`, then `nn.Linear(128, V)` (with bias, not tied to the embedding)
Initialisation	PyTorch defaults
Forward returns	logits of shape (B, T, V) and a list with each block's attention weights, each of shape $(B, 4, T, T)$

8. Training and evaluation

- Optimizer: `torch.optim.Adam(model.parameters(), lr=1e-3)` with all other arguments at their defaults. No scheduler, no gradient clipping, no weight decay.
- Loss: `nn.CrossEntropyLoss(ignore_index=0)` on logits reshaped to $(B{\cdot}T, V)$ and targets reshaped to $(B \cdot T)$.
- Epochs: 20. After every epoch, record the training loss (the mean of that epoch's batch losses) and the validation loss.
- Validation and test loss: run in `model.eval()` under `torch.no_grad()`, sum the cross-entropy over all non-`<pad>` target tokens and divide by the number of those tokens.
- Perplexity = exp(loss). Accuracy = share of non-`<pad>` target tokens where the arg-max of the logits equals the target.

9. Time and marks

Task	Title	Marks	Minutes
1	Open the Box	6	20
2	Learn to Look Back	12	35
3	Raise the Apprentice	9	35
4	Detective Work	12	40
5	The Story Thermostat	7	25
6	The Blindfold Test	4	15
	Final check of the notebook		10
Total		**50**	**180**

Tip: start training in Task 3 as soon as your model runs, and work on the next task while it trains. Each model now trains for 20 epochs, and Task 6 trains a second model, so start both training runs as early as you can.

Task 1 Open the Box — 6 Marks

> *The librarian wheels out a cart of well-worn stories. Before the apprentice can tell a story, it has to learn to read one.*

Implement settings 2 to 6, then report:

1. The number of non-empty stories in the file, the number left after the 190-token filter, the sizes of the three subsets, and the vocabulary size V. **2 marks**
2. The percentage of word tokens in the encoded validation set (all tokens of each validation story, specials excluded) that are `<unk>`. **2 marks**
3. For story 0 of the training set: its first 15 tokens, the first 16 values of its encoded sequence x, and the number of `True` values in its padding mask. **2 marks**

Task 2 Learn to Look Back — 12 Marks

> *A story only makes sense if you remember what already happened. But the apprentice must obey one rule: it may look back at what has been said, never ahead at what has not.*

Implement `CausalSelfAttention(d_model=128, n_heads=4)` as an `nn.Module`. You may use `nn.Linear`, `torch.softmax` and `torch.tril`. You may not use `nn.MultiheadAttention` or `F.scaled_dot_product_attention`.

Specification

6 marks

1. Layers: three `nn.Linear(128, 128)` for Q, K, V and one `nn.Linear(128, 128)` output projection W_O, all with bias.

2. `forward(h, pad_mask, head_mask=None)`, where `h` has shape $(B, T, 128)$ and `pad_mask` has shape (B, T).
3. Split Q, K, V into 4 heads of 32 features. For each head ℓ:

$$S_\ell = \frac{Q_\ell K_\ell^\top}{\sqrt{32}}, \qquad A_\ell = \text{softmax}\big(S_\ell \text{ with blocked entries set to } -\infty\big), \qquad O_\ell = A_\ell V_\ell.$$

   Entry (i, j) is allowed only if $j \le i$ *and* key position j is a real token. Because padding is always at the end, every row keeps at least position 0, so no row is entirely blocked.
4. `head_mask` is an optional tensor of shape $(4,)$ (default: all ones). Multiply each head's output O_ℓ by `head_mask[`ℓ`]` before joining the heads. You need this in Task 4.
5. Join the heads back to $(B, T, 128)$, apply W_O, and return `(output, A)` with A of shape $(B, 4, T, T)$. Do not apply dropout to A.

Checks

6 marks Call `torch.manual_seed(42)`, create the module, put it in `eval()` mode, and create `h = torch.randn(B, 191, 128)` where B is the size of the first validation batch. Use that batch's `inp != 0` as the padding mask. Report:

1. The minimum and maximum row sum of A over rows whose query position is a real token. 2 marks
2. The largest weight on any future position ($j > i$) and the largest weight on any padding key. Both must be exactly 0. 2 marks
3. **Tamper test:** replace `h[:, 11:, :]` with new random values (same padding mask) and run the module again. Report the largest absolute difference between the two outputs at positions 0 to 10. It must be 0 up to floating-point error. 2 marks

Task 3 Raise the Apprentice — 9 Marks

> *Listening is not yet storytelling. The apprentice also needs somewhere to think and a voice to speak with. Then it reads, and reads, and reads.*

1. **Build it** 2 marks. Implement the model exactly as in setting 7, with a constructor flag `use_pos=True` (you need `False` in Task 6). Report the total number of trainable parameters.
2. **Train it** 3 marks. Train with the settings in setting 8 and print the training and validation loss after every epoch.
3. **Plot it** 2 marks. Plot training and validation loss against epoch (1 to 20) on one figure with a legend.
4. **Test it** 2 marks. Report test loss, test perplexity and test accuracy.

Bonus +4

Build a bigram model from the encoded training data: count every pair `(inp[i], tgt[i])` with `tgt[i]` $\neq 0$ and use

$$P(w \mid v) = \frac{c(v, w) + 1}{c(v) + V}.$$

Report its perplexity on the non-`<pad>` test targets next to your transformer's test perplexity.

Task 4 Detective Work: What Does It Remember? — 12 Marks

> *Stories keep reusing the same names and things: "Lily … Lily". Predicting such a callback needs memory, and memory is what attention is for. Time to find out which of the apprentice's heads is doing the remembering.*

Use your trained Task 3 model in `eval()` mode on the test set.

Definitions

Treat IDs 4 to 53 (the 50 most frequent tokens) as *common*. In a test sequence, position i ($0 \le i \le 190$) is a **callback position** if

- `tgt[i]` ≥ 54 (not special, not `<unk>`, not common), and
- `tgt[i]` appears somewhere in `inp[1]`, …, `inp[i]`.

Every other position with `tgt[i]` $\neq 0$ is an **other position**. The attention row of position i is what helps predict `tgt[i]`.

1. **Is it harder?** (3 marks). Using per-token cross-entropy, report the number of callback positions and other positions, and the mean loss at each.

2. **Score the heads** (4 marks). For block $b \in \{1, 2\}$ and head $\ell \in \{1, 2, 3, 4\}$, the callback score is

$$\text{score}(b, \ell) = \operatorname*{mean}_{\text{callback } i} \sum_{\substack{1 \le j \le i \\ \texttt{inp[j]=tgt[i]}}} A_\ell^{(b)}[i, j].$$

   Show the eight scores as a 2×4 heatmap (rows = blocks, columns = heads) with each value written in its cell. Name the best head (highest score).

3. **Watch it work** (2 marks). Take test story 0 (if it has fewer than 2 callback positions among positions 0 to 24, use the first test story that has at least 2). Plot the best head's weights for positions 0 to 24 as a 25×25 heatmap: query tokens on the y-axis, key tokens on the x-axis, both labelled with the words from `inp[0:25]`. Add a $*$ to the y-label of each callback position.

4. **Head surgery** (3 marks). Using `head_mask`, switch off (a) the best head and, separately, (b) the head with the lowest score, as a control. Report a table with three rows (nothing removed, best head removed, control head removed) and two columns (test loss over all non-`<pad>` targets, mean loss at callback positions).

Task 5 The Story Thermostat — 7 Marks

> *At last the lamp is lit. You hand the apprentice the first few words of a tale and step back. How careful, or how daring, should its voice be?*

Generation rules

- Prompts: for each of the first 50 test stories, the prompt is `inp[0:7]`, i.e. `<bos>` plus the first 6 tokens.
- Loop: run the model in `eval()` mode under `torch.no_grad()` on the sequence so far, take the logits at the last position, set the logits of IDs 0, 1 and 3 to $-\infty$, choose the next token, and append it. Stop when `<eos>` is chosen or the sequence (including `<bos>`) reaches 191 tokens.
- Settings: **greedy** (arg-max); **temperature 0.7** and **temperature 1.2**, sampling with `torch.multinomial(torch.softmax(logits / T, -1), 1)`.
- Call `torch.manual_seed(42)` once immediately before generating the 50 stories of each setting, and generate them in order, story 0 first.
- The *generated part* of a story is the tokens after the prompt, without `<eos>`. Show text by joining tokens with single spaces.

1. **Generate** (4 marks). All three settings working on all 50 prompts.
2. **Compare** (2 marks). One table with a row per setting and three columns: mean length of the generated part (in tokens), the percentage of stories that ended with `<eos>`, and distinct-2. Distinct-2 is the number of unique adjacent token pairs divided by the total number of adjacent token pairs, counted over the generated parts of all 50 stories together.
3. **Read them** (1 mark). Print the prompt and generated part for prompt 0 under all three settings.

Task 6 The Blindfold Test — 4 Marks

> *What happens if the apprentice can no longer tell where in the sentence each word sits?*

1. Call `torch.manual_seed(42)`, build the model with `use_pos=False` (no positional embedding, so $h = \text{Dropout}(0.1)\,(\text{token emb})$), and train it with exactly the same settings as in Task 3. (2 marks)
2. Report one table with a row per model (with and without positions) and four columns: test loss, test perplexity, test accuracy, and mean loss at callback positions. Plot the validation loss of both models against epoch on one figure. (2 marks)

Hand In

Submit one `.ipynb` notebook with a heading for each task. Before submitting, restart the kernel and run all cells from top to bottom so that every output is visible. All reported numbers and tables must be produced by code, and every heatmap must have word labels on its axes. No written explanations are required.